

WIDS 2026 Report

Ask the Image: Explainable AI for Visual Q and A

Siddharth Jeedula

Contents

1	Introduction	3
2	Week 1: Computer Vision Foundations	3
2.1	Overview	3
2.2	Environment and Tooling	3
2.3	Image Representation and Preprocessing	4
2.4	Neural Networks and CNNs	4
2.5	Transfer Learning	4
3	Week 2: Natural Language Processing Foundations	5
3.1	Overview	5
3.2	Text Processing Pipeline	5
3.3	Tokenization Methods	5
3.4	Vectorization Techniques	5
3.4.1	Bag of Words	5
3.4.2	TF-IDF	6
3.5	Word Embeddings	6
3.6	Regex and NLP Libraries	6
4	Week 3: Multimodal Learning and Explainability	6
4.1	Overview	6
4.2	Encoder–Decoder Architecture	6
4.3	Attention Mechanisms	7
4.4	Vision Transformers	7
4.5	Explainable AI Techniques	7
4.6	Grad-CAM	7

5	Final Project: Image Captioning System	7
5.1	Objective	7
5.2	Dataset	7
5.3	Model Architecture	8
5.3.1	Encoder	8
5.3.2	Decoder	8
5.4	Text Processing	8
5.5	Training Setup	8
6	Experiments and Comparisons	9
7	Evaluation Metrics	9
7.1	BLEU Score	9
7.2	METEOR Score	9
7.3	Caption Statistics	9
8	Qualitative Analysis	9
9	Limitations	10
10	Future Improvements	10
11	Conclusion	10

1 Introduction

The WIDS 2026 program focuses on building strong theoretical and practical foundations in Computer Vision (CV) and Natural Language Processing (NLP), followed by their integration into multimodal systems. This report documents week-wise learning progress, practical implementations, and a final project on image captioning using a CNN encoder and Transformer decoder.

The learning progression was structured in three major stages. The first stage focused on computer vision fundamentals and deep learning workflows. The second stage introduced natural language processing techniques and classical text representation methods. The third stage connected both domains through multimodal learning, interpretability, and image captioning.

The final project integrates these concepts into a complete end-to-end caption generation system trained on a real-world dataset.

2 Week 1: Computer Vision Foundations

2.1 Overview

Week 1 introduced the fundamental principles of computer vision and the tools required to process visual data using deep learning frameworks. The goal was to understand how images are represented numerically and how models learn patterns from pixel data.

The workflow emphasized both theoretical understanding and practical implementation using modern libraries.

2.2 Environment and Tooling

The development environments used included:

- Python for model development and experimentation
- Google Colab for GPU-based execution
- Local Conda environments with GPU support
- PyTorch for deep learning model implementation
- OpenCV for classical image processing

GPU acceleration played a key role in enabling efficient model training and experimentation.

2.3 Image Representation and Preprocessing

Images are represented as multi-dimensional tensors where each pixel contains intensity values across channels. Preprocessing steps are critical to ensure model stability and performance.

Key preprocessing operations included:

- Image resizing to fixed dimensions
- Normalization using dataset statistics
- Data augmentation for generalization
- Tensor conversion and batching

These steps reduce noise, standardize input ranges, and improve training convergence.

2.4 Neural Networks and CNNs

Convolutional Neural Networks (CNNs) were studied as the primary architecture for image tasks. Core concepts included:

- Convolutional filters for feature extraction
- Pooling layers for spatial reduction
- Activation functions for non-linearity
- Fully connected layers for classification

CNNs learn hierarchical features, where early layers capture edges and textures while deeper layers capture semantic patterns.

2.5 Transfer Learning

Transfer learning was introduced as a method to reuse pretrained models for new tasks. Instead of training from scratch, pretrained backbones can be adapted by replacing the final classification layer.

Benefits include:

- Reduced training time
- Better performance on small datasets
- Improved feature generalization

Experiments were conducted using pretrained backbones with fine-tuned output layers.

3 Week 2: Natural Language Processing Foundations

3.1 Overview

Week 2 focused on Natural Language Processing and how machines convert raw text into structured representations suitable for modeling. Emphasis was placed on preprocessing pipelines and classical feature extraction techniques.

3.2 Text Processing Pipeline

Raw text must be cleaned and standardized before modeling. The preprocessing pipeline included:

- Lowercasing
- Punctuation removal
- Tokenization
- Stopword removal
- Stemming and lemmatization

These steps reduce vocabulary size and remove noise.

3.3 Tokenization Methods

Different tokenization strategies were studied:

- Word-level tokenization
- Sentence tokenization
- Subword tokenization

Subword tokenization helps handle rare and unseen words.

3.4 Vectorization Techniques

Classical text representation techniques were implemented:

3.4.1 Bag of Words

Bag of Words represents documents using word frequency counts. It ignores order but captures presence information.

3.4.2 TF-IDF

TF-IDF improves upon Bag of Words by weighting words according to importance across documents.

$$TF-IDF = TF \times \log \left(\frac{N}{DF} \right)$$

This reduces the impact of very common words.

3.5 Word Embeddings

Word embeddings were introduced to capture semantic similarity using dense vectors. The key idea is that words appearing in similar contexts have similar vector representations.

Conceptual understanding of embedding models was developed.

3.6 Regex and NLP Libraries

Regular expressions were used for text cleaning and pattern matching. NLP libraries were used for tokenization and linguistic processing tasks.

4 Week 3: Multimodal Learning and Explainability

4.1 Overview

Week 3 bridged computer vision and NLP through multimodal learning systems such as image captioning. The focus expanded to encoder-decoder models, attention mechanisms, and explainable AI.

4.2 Encoder–Decoder Architecture

Image captioning models use:

- An image encoder to extract visual features
- A language decoder to generate captions

The encoder produces feature embeddings which are consumed by the decoder to generate text sequentially.

4.3 Attention Mechanisms

Attention allows models to focus on relevant image regions while generating each word. This improves caption quality and interpretability.

Attention provides alignment between visual features and generated tokens.

4.4 Vision Transformers

Transformer-based encoders process images as patch sequences rather than convolutional grids. This enables global context modeling.

The conceptual difference between CNN and Transformer encoders was analyzed.

4.5 Explainable AI Techniques

Model interpretability techniques were implemented to understand CNN behavior.

Methods included:

- Filter visualization
- Feature map visualization
- Gradient-based attribution
- Activation heatmaps

4.6 Grad-CAM

Grad-CAM was implemented to visualize which image regions influenced model predictions. Heatmaps were generated for both correct and incorrect predictions to study model behavior.

5 Final Project: Image Captioning System

5.1 Objective

The final project aimed to build an end-to-end image captioning model that generates descriptive sentences for images using a CNN encoder and Transformer decoder.

5.2 Dataset

The Flickr8k dataset was used. It contains approximately 8000 images with five captions per image. The dataset provides diversity but is relatively small, making transfer learning important.

5.3 Model Architecture

5.3.1 Encoder

The encoder used a pretrained CNN backbone with the classification layer removed. Global pooled features were extracted as embeddings.

5.3.2 Decoder

The decoder was a Transformer-based architecture with:

- Multiple decoder layers
- Multi-head attention
- Positional encoding
- Causal masking

Teacher forcing was used during training.

5.4 Text Processing

Captions were tokenized at the word level. Special tokens were added:

- Beginning token
- End token
- Padding token
- Unknown token

Caption length was limited to a fixed maximum value.

5.5 Training Setup

Training configuration included:

- Cross-entropy loss
- Padding token ignored in loss
- Adam optimizer
- Learning rate scheduling
- Fixed random seed

Feature caching was used to speed up repeated training runs.

6 Experiments and Comparisons

Backbone comparisons were performed using different CNN encoders. Performance differences were analyzed in terms of caption quality and training efficiency.

Decoder behavior under greedy decoding was studied and compared across settings.

7 Evaluation Metrics

Multiple evaluation metrics were used.

7.1 BLEU Score

BLEU measures n-gram overlap between generated and reference captions.

7.2 METEOR Score

METEOR considers synonym and recall alignment, making it more flexible.

7.3 Caption Statistics

Additional analysis included:

- Caption length distribution
- Repetition percentage
- Vocabulary coverage

8 Qualitative Analysis

Qualitative evaluation was conducted using:

- Successful caption examples
- Failure cases
- Misclassification analysis
- Attention visualization
- Grad-CAM heatmaps

This helped identify strengths and weaknesses in model behavior.

9 Limitations

Several limitations were identified:

- Small dataset size
- Limited vocabulary coverage
- Loss of spatial detail due to pooling
- Greedy decoding reduces diversity
- Caption bias toward frequent patterns

10 Future Improvements

Possible improvements include:

- Larger caption datasets
- Subword tokenization
- Beam search decoding
- Transformer-based image encoders
- Stronger augmentation
- Reinforcement learning fine-tuning

11 Conclusion

The WIDS 2026 learning journey provided a structured progression from computer vision fundamentals to natural language processing and multimodal systems. Practical implementations reinforced theoretical concepts across CNNs, NLP preprocessing, attention mechanisms, and interpretability.

The final image captioning project demonstrated how visual and textual models can be combined into a functional system. The experiments, evaluations, and analyses provided insight into both model capability and limitations.

This foundation supports further exploration into advanced multimodal AI systems and research-level model development.